Experiment 6

Name : Saurabh Kumar **UID :**23BCS10675
Branch: BE-CSE **Section/Group:** KRG-3B
Semester: 6th **Date:** 03/03/2026
Subject Name: Full Stack-II **Subject Code:** 23CSH-309

Aim:

To design and implement a Spring Boot REST API using Layered Architecture, applying Inversion of Control (IoC) and Dependency Injection (DI) to build a structured backend for the HealthHub application, including CRUD operations, DTO mapping, validation, and global exception handling.

Objective:-

• Understand the fundamentals of Spring Boot and its role in simplifying backend application development, and learn how to configure and run a basic Spring Boot application.

• Apply the concepts of Inversion of Control (IoC) and Dependency Injection (DI) in Spring, and utilize annotations such as @Autowired, @Service, and @Repository to manage dependencies effectively.

• Implement Layered Architecture by structuring the application into Controller, Service, and Repository layers, ensuring proper separation of concerns in backend design.

• Develop RESTful APIs using Spring Boot by creating endpoints that perform CRUD operations through appropriate HTTP methods such as GET, POST, PUT, and DELETE.

• Use Data Transfer Objects (DTOs) to facilitate communication between the API and the application, ensuring separation between internal data models and external responses.

• Apply input validation using Jakarta Validation annotations such as @NotNull and @Email to maintain data accuracy and integrity.

• Implement global exception handling using @ControllerAdvice to manage errors centrally and improve application reliability.

• Build backend functionality for the HealthHub patient management system, enabling the storage, retrieval, updating, and deletion of patient records.

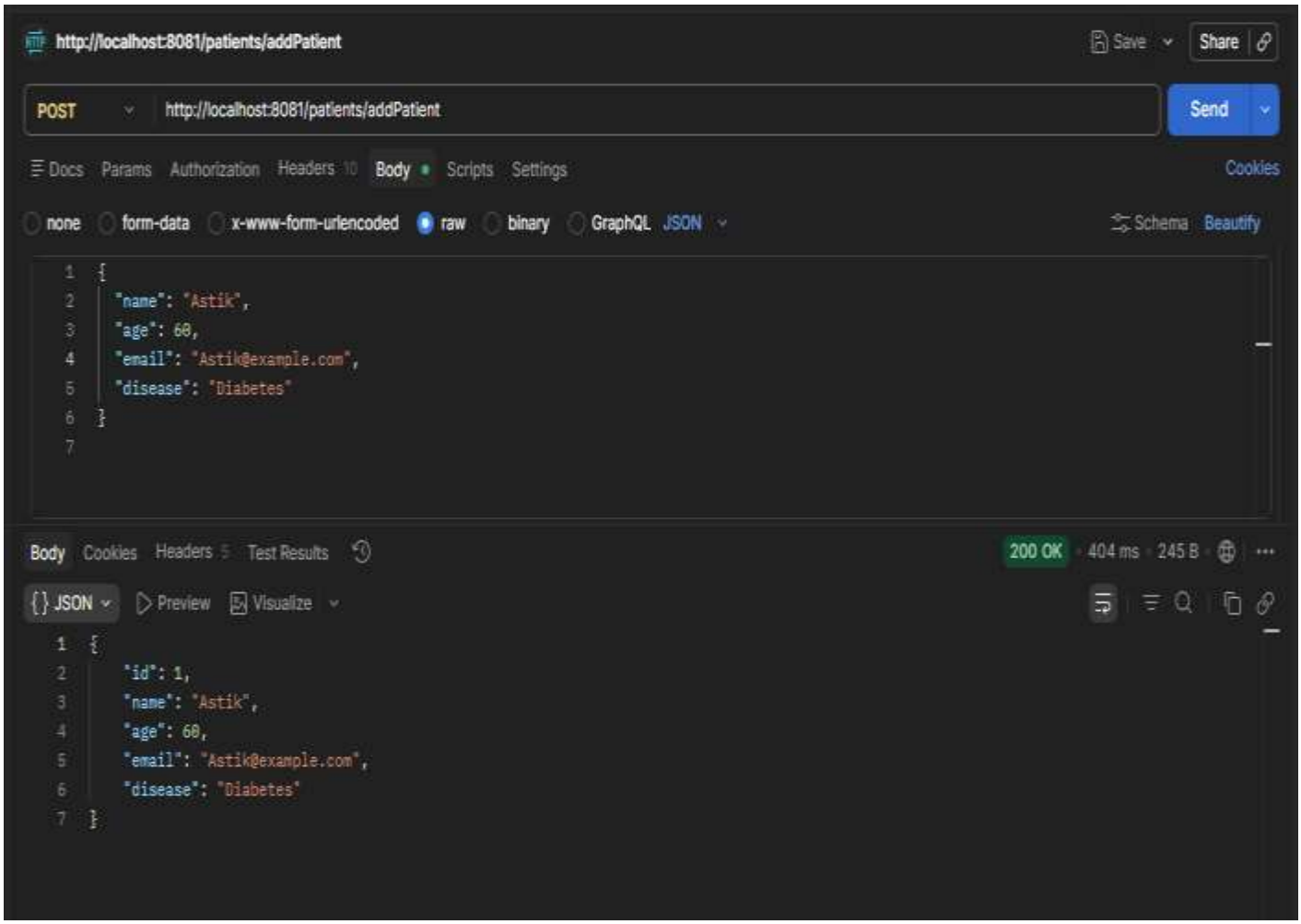
Implementation/Code:

The Healthhub backend is built using Spring Boot, providing a robust and scalable RESTful API architecture. It utilizes a layered design—featuring Controllers, Services, and Repositories—to ensure proper separation of concerns. Controllers, such as the PatientController, handle incoming HTTP requests and map them to business logic operations in the Service layer, while Data Transfer Objects (DTOs) and validation annotations (@Valid) ensure that incoming data is structured and secure. This architecture allows Healthhub to efficiently process patient records, handle database transactions, and serve data to the frontend in a reliable manner.

STRUCTURE:

Output:

- Spring Boot application started successfully on localhost:8080
- **POST** /patients/addPatient API successfully added new patient records to the database
- **GET** /patients/getPatients API returned the complete list of patients
- **GET** /patients/{id} API returned specific patient details based on their ID
- **PUT** /patients/updatePatient/{id} API successfully updated existing patient details
- **DELETE** /patients/{id} API successfully removed patient data from the system
- Postman confirmed successful API responses with Status 200 OK
- Layered architecture (Controller-Service-Repository) ensured separation of concerns and clean backend design



Discover. Learn. Empower.

http://localhost:8081/patients/getPatients

Save Share

GET http://localhost:8081/patients/getPatients Send

Docs Params Authorization Headers 10 Body Scripts Settings Cookies

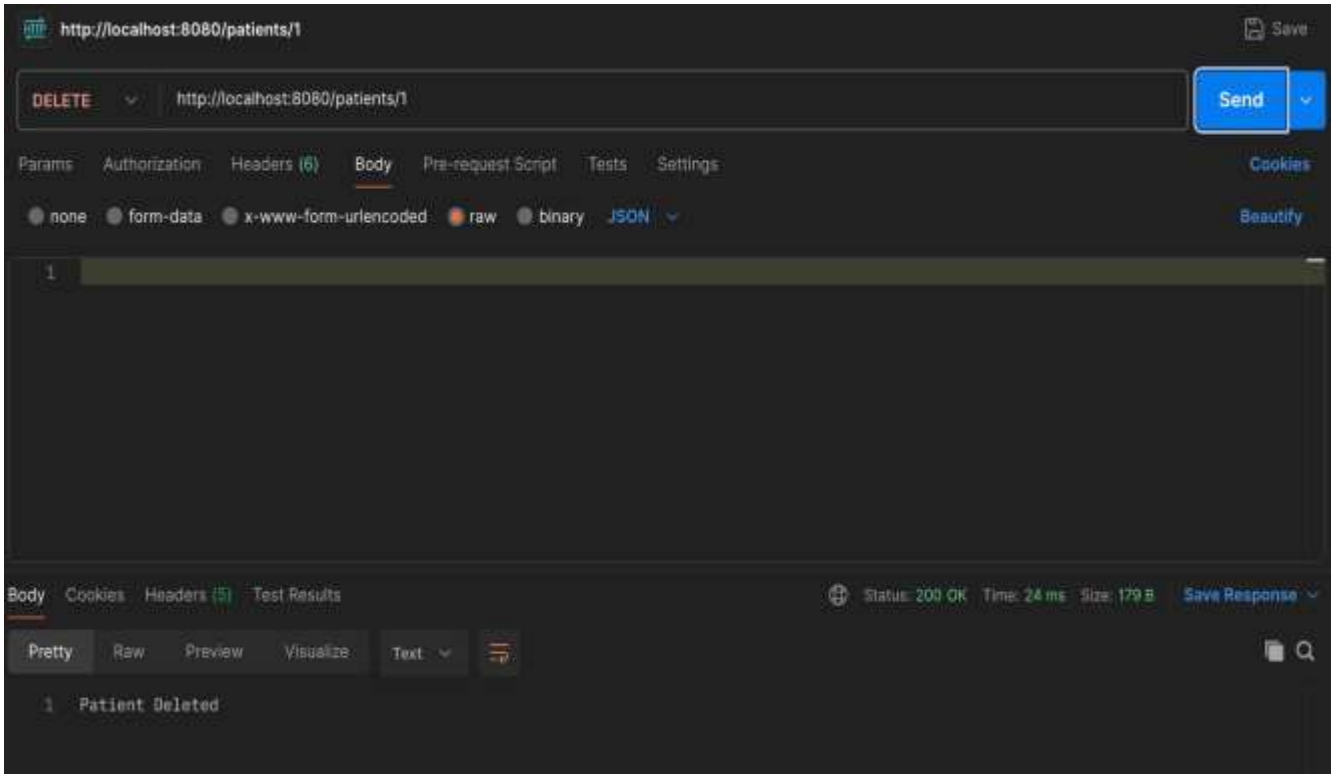
none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

Body Cookies Headers 5 Test Results 200 OK 225 ms 340 B

{} JSON Preview Visualize


```
[
    {
        "id": 1,
        "name": "Astik",
        "age": 68,
        "email": "Astik@example.com",
        "disease": "Diabetes"
    },
    {
        "id": 2,
        "name": "Someone",
        "age": 67,
        "email": "Someone@example.com",
        "disease": "Mental-Disorder"
    }
]
```

Discover. Learn. Empower.



5. Learning Outcomes (What I Have Learnt):

- Understanding of **Spring Boot backend development**
- Implementation of **Layered Architecture (Controller, Service, Repository)**
- Usage of **Dependency Injection in Spring Boot**
- Development of **RESTful APIs using HTTP methods**
- Implementation of **CRUD operations in backend applications**
- Understanding of **DTO mapping for API communication**
- Applying **validation annotations for data integrity**
- Handling errors globally using **Spring Exception Handling** • Testing APIs using **Postman**